Software Engineering

BCSE301L

Module 2

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Project Planning

- The *process* of *defining and outlining* the *scope, goals, timelines, resources, and budget* for a software development project.

- It involves creating a detailed plan that *guides* the *project team* through the *development process*, ensuring that the project is *completed on time, within budget, and to the required quality standards*.

Project Planning – Key Elements

Project Scope	• Defining what the project will deliver and what is excluded
Goals and Objectives	• Establishing clear project goals and objectives
Timelines	• Creating a project schedule, including milestones and deadlines.
Resources	• Identifying and allocating the necessary resources, including personnel, equipment, and materials.

Project Planning – Key Elements

Budget

- Establishing a budget for the project, including costs for resources, equipment, and services.

Risk Management

- Identifying potential risks and developing strategies to mitigate them.

Quality Management

- Defining quality standards and processes to ensure the project meets the required quality levels.Creating a project schedule, including milestones and deadlines.

Project Plan

- A project plan is a *detailed document* that *outlines* the *objectives, scope, timelines, resources*, and *budget* for a specific project.

- It serves as a *roadmap* for the *project team, stakeholders*, and *sponsors* to ensure everyone is on the same page and working towards the same goals.

Project Plan – Key Components

Project Scope

- Defines what the project will deliver and what is excluded.

Objectives

- Clearly states the project's goals and objectives.

Timelines

- Establishes the project schedule, including milestones and deadlines.

Resources

- Identifies the necessary resources, including personnel, equipment, and materials.

Project Plan – Key Components

Budget

- Establishes the project budget, including costs for resources, equipment, and services.

Risk Management

- Identifies potential risks and outlines strategies for mitigation.

Quality Management

- Defines quality standards and processes to ensure the project meets the required quality levels.

Communication

- Outlines the communication plan, including stakeholder engagement and reporting requirements

Project Plan – Key Components

Introduction	Project Organization	Risk Analysis	Software Resource
• Project objectives, constraints, and scope.	• Team structure, roles, and responsibilities.	• Identification of potential risks, likelihood, and mitigation strategies.	• Specification of necessary resources and estimated costs.

Project Plan – Key Components

Work Breakdown	Project Schedule	Monitoring and Reporting Mechanisms
• Division of project into activities, inputs, and outputs.	• Dependencies, timelines, milestones, and resource allocation.	• Definition of management reports and project monitoring processes.

Project Plan – Benefits

Improved Project Control	A clear plan helps the project team stay focused and on track.
Better Resource Allocation	A plan ensures that resources are allocated efficiently and effectively.
Reduced Risk	Identifying potential risks and developing mitigation strategies reduces the likelihood of project failure.
Increased Customer Satisfaction	A well-planned project ensures that the final product meets the customer's requirements and expectations.
Improved Team Collaboration	A clear plan helps team members understand their roles and responsibilities, promoting better collaboration and communication.

Project Planning Process

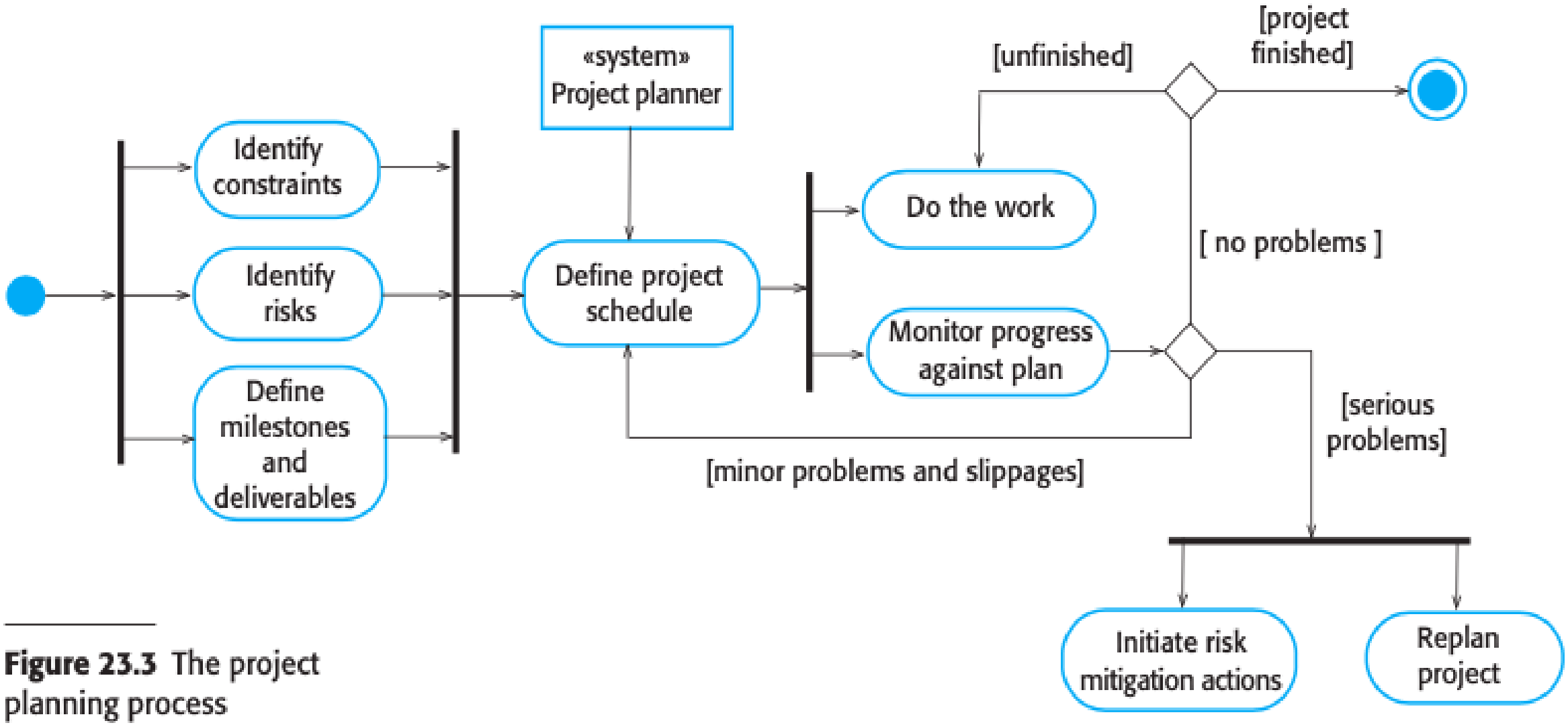


Figure 23.3 The project planning process

Project Planning Process

- **Create Initial Plan:** Create an initial project plan, including constraints, milestones, and deliverables. This plan should be based on realistic assumptions and take into account potential risks and contingencies.
- **Assess Constraints:** Assess the project constraints, such as:
  - Delivery date
  - Staff availability
  - Budget
  - Tools and resources

Project Planning Process

- **Identify Milestones and Deliverables:** Identify the project milestones and deliverables, such as:
  - Key project events (e.g., completion of requirements gathering)
  - Deliverables (e.g., software, documentation)
- **Create Schedule:** Create a schedule that outlines the project timeline, including:
  - Start and end dates
  - Dependencies between tasks
  - Resource allocation

Project Planning Process

- **Initiate Activities**: Initiate the project activities, such as:
  - Requirements gathering
  - Design
  - Development
  - Testing
- **Monitor Progress**: Monitor the project progress, including:
  - Tracking task completion
  - Identifying and addressing delays
  - Reviewing and revising the project plan as needed

Project Planning Process

- **Review and Revise:** Regularly review the project progress and revise the project plan as needed. This includes:
  - Updating the schedule
  - Reallocating resources
  - Addressing changes in project scope or requirements

Project Planning Process - Overall View

Iterative Process

- Project planning is an ongoing process that starts during the project startup phase and continues throughout the project.

Initial Plan

- Create an initial project plan, including constraints, milestones, and deliverables.

Assess Constraints

- Identify project constraints, such as delivery date, staff, budget, and tools.

Identify Milestones and Deliverables

- Determine project milestones and deliverables.

Schedule and Monitor

- Create a schedule, initiate activities, and monitor progress.

Review and Revise

- Regularly review progress, note discrepancies, and revise the plan as needed

Milestones

- Project milestones are significant events or accomplishments that mark important stages in a project's lifecycle.

- They represent key achievements or completion of major tasks, and are used to *measure progress, track performance*, and *make adjustments* to the project plan as needed.

Milestones - Characteristics

- Specific: Clearly defined and easy to understand.
- Measurable: Quantifiable, allowing for objective measurement of progress.
- Achievable: Realistic and attainable, given the project's resources and constraints.
- Relevant: Aligns with the project's goals and objectives.
- Time-bound: Has a specific deadline or timeframe for completion

Milestones - Examples

- Completion of Requirements Gathering: Finalizing the project's requirements and specifications.
- Design Review: Completing the design phase and receiving approval from stakeholders.
- Alpha/Beta Release: Releasing a preliminary version of the product for testing and feedback.
- Final Product Delivery: Completing the final product and delivering it to the customer.
- Project Closure: Formalizing the project's completion and documenting lessons learned.

Deliverables

- Project deliverables are the *tangible or intangible products* or services that are *produced and delivered* as a result of a project.

- They are the specific outcomes or results that the project is intended to achieve and are usually defined in the project scope statement.

Deliverables - Types

Tangible Deliverables - Physical products or documents, such as:

- Software code
- Hardware components
- Reports
- Designs

Intangible Deliverables -Non-physical outcomes or services, such as:

- Training
- Consulting
- Support
- Maintenance

Deliverables - Characteristics

- **Specific:** Clearly defined and easy to understand.
- **Measurable:** Quantifiable, allowing for objective measurement of progress.
- **Achievable**: Realistic and attainable, given the project's resources and constraints.
- **Relevant:** Aligns with the project's goals and objectives.
- **Time-bound:** Has a specific deadline or timeframe for completion.

Project Scope

- Project scope refers to the ***specific goals, objectives, and deliverables*** that a project aims ***to achieve***.

- It defines *what is included and excluded from the project and* provides a clear understanding of what the project will deliver.

Project Scope - Components

Project Objectives

- Specific, measurable, achievable, relevant, and time-bound (SMART) goals that the project aims to accomplish.

Deliverables

- Tangible or intangible products or services that the project will produce.

Boundaries

- Clear definitions of what is included and excluded from the project.

Assumptions

- Statements that are assumed to be true for the project to succeed.

Constraints

- Factors that limit the project's scope, such as budget, time, or resources

Project Scope – Example E-commerce Website

- **Project Objectives:**
  - Develop a new e-commerce website that allows customers to browse and purchase products online.
  - Improve the overall user experience and increase customer engagement.
  - Increase online sales by 20% within the first six months of launching the new website.

Project Scope – Example E-commerce Website

- **Deliverables:**
  - A fully functional e-commerce website with the following features:
    - Product catalog
    - Shopping cart
    - Payment gateway
    - User registration and login
  - A comprehensive user manual and training documentation for website administrators.
  - A project report detailing the development process, challenges, and lessons learned.

Project Scope – Example E-commerce Website

- **Boundaries:**
  - The project scope excludes the development of a mobile app.
  - The project scope excludes the integration of third-party services, such as social media or email marketing.
  - The project scope excludes the provision of ongoing website maintenance and support.

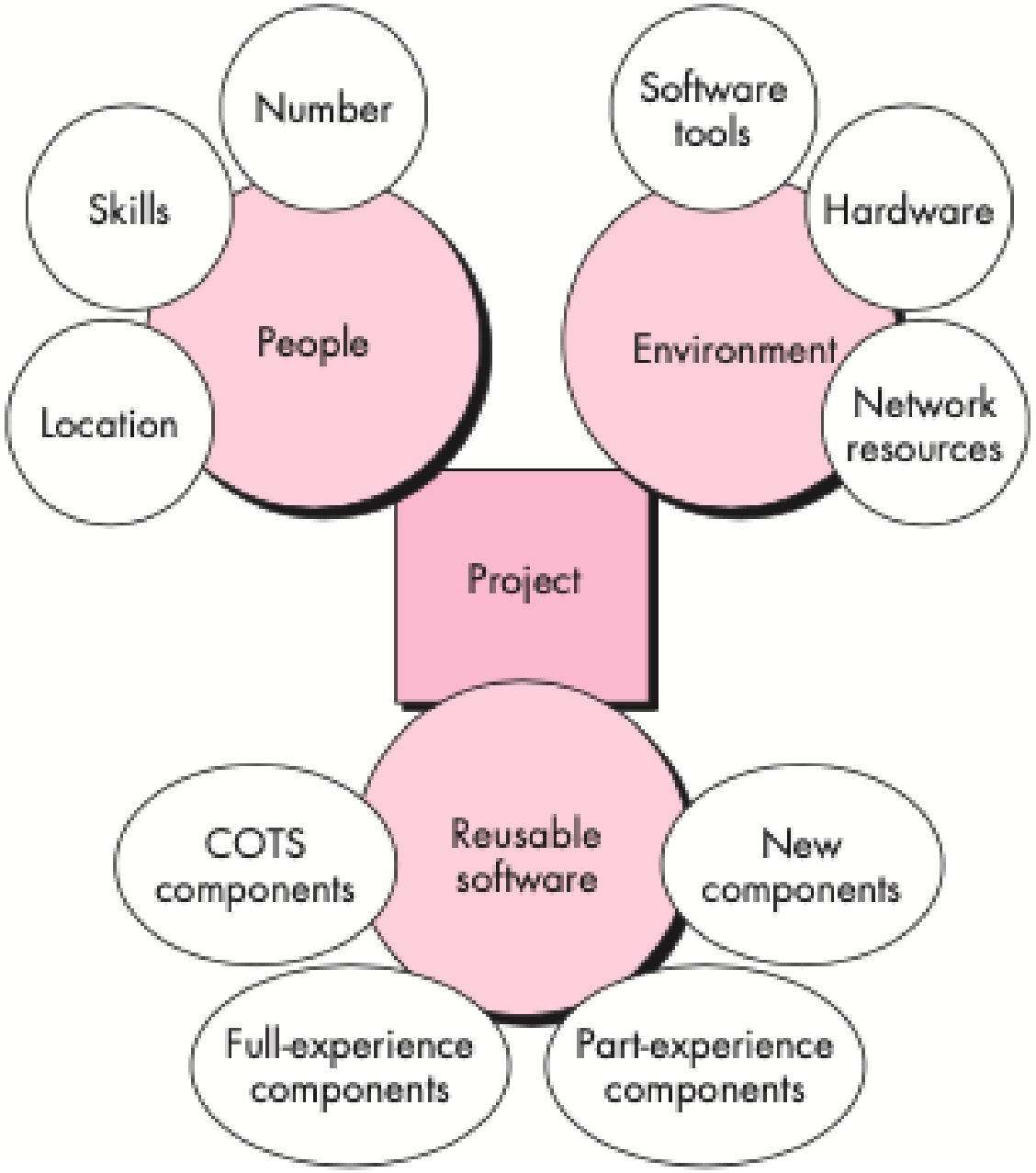
Project Scope – Example E-commerce Website

- **Assumptions:**
  - The project assumes that the client will provide all necessary content, including product information and images.
  - The project assumes that the client will provide access to their existing website and database.
  - The project assumes that the client will approve all design and functionality decisions in a timely manner.

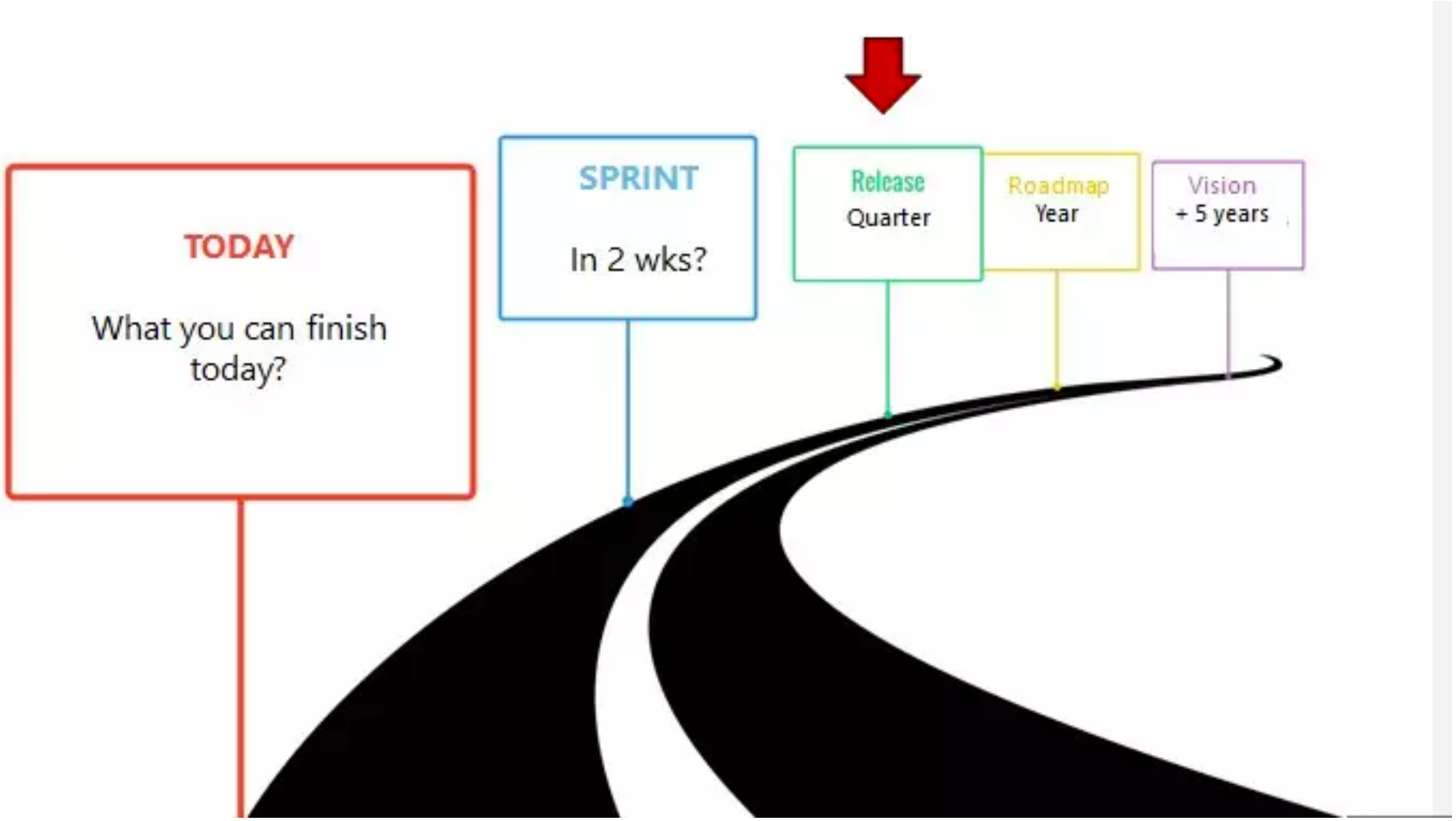
Project Scope – Example E-commerce Website

- **Constraints:**
  - The project must be completed within six months.
  - The project budget is $100,000.
  - The project team will consist of a project manager, two developers, and a designer.

Resources


Number
Skills
People
Location
Software tools
Hardware
Environment
Network resources
Project
Reusable software
COTS components
New components
Full-experience components
Part-experience components

Agile Methodology Planning



Agile Methodology Planning

Agile Methodology Planning

Agile Methodology Planning

- **User Stories**: The foundation of the planning game. These are short, simple descriptions of user needs in the form "As a <type of user>, I want <some goal> so that <some reason>."
- **Planning Game**: An iterative process involving both release planning and iteration planning, driven by user stories.
- **Story Ranking**: Stories are prioritized based on their complexity and estimated effort, often through relative ranking (comparing stories in pairs).
- **Effort Points**: Notional points assigned to stories to represent their relative effort (e.g., 8 points for a complex story, 2 points for a simple one).
- **Velocity**: The number of effort points a team can typically complete in a given time period (e.g., per day or per iteration). Used to estimate overall project effort.

Agile Methodology Planning

- **Release Planning:**
  - Selects and refines user stories for a major system release.
  - Involves customer collaboration to determine release scope and timeline.
- **Iteration Planning:**
  - Focuses on planning the next increment of the system (typically 2-3 weeks).
  - Chooses stories to be implemented based on velocity and time constraints.
- **Task Planning:**
  - Breaks down user stories into smaller, manageable development tasks (4-16 hours each).
  - Allows developers to self-select tasks, promoting ownership and motivation.

Agile Methodology Planning

- **Mid-Iteration Review:**
  - **Progress Tracking**: Monitors story point progress (should be 50% complete halfway through).
  - **Scope Adjustment**: If progress falls behind, stories are removed to ensure on-time delivery, even if it impacts customer plans.

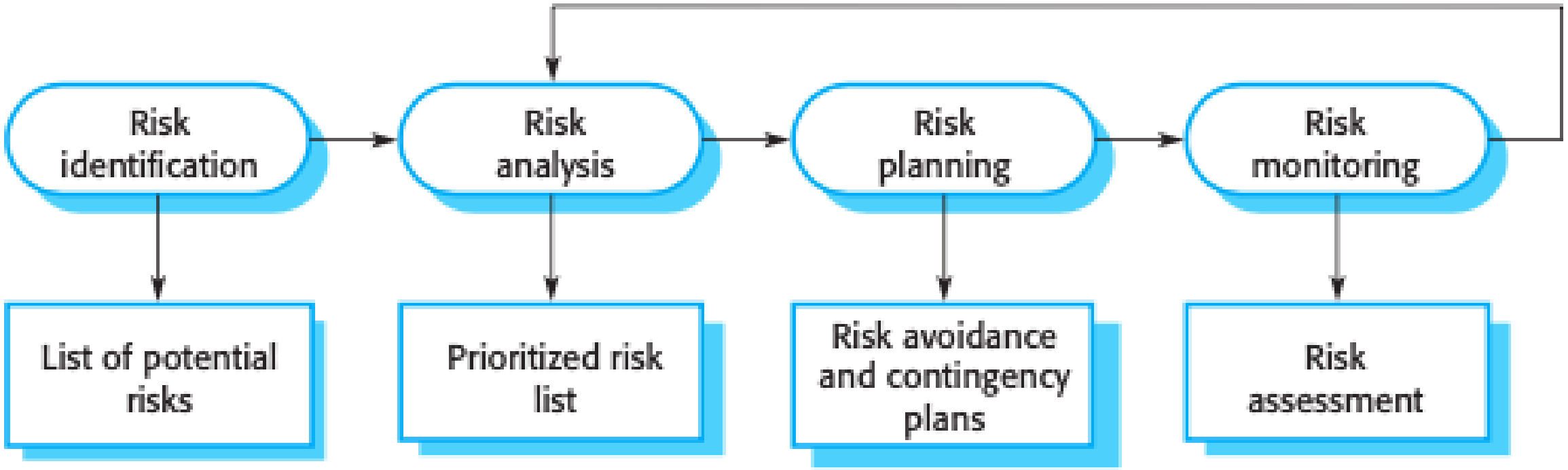
Risk Management

- A **risk** is any ***event or circumstance*** that could ***negatively impact*** the project, the software, or the organization.
- Risk Categories:
  - **Type of Risk**: Technical, organizational, etc.
  - **Impact Area**: Project (schedule/resources), Product (quality/performance), Business (organizational impact).
  - **Project Risk Example**: Loss of a key personnel (system architect). Impacts project schedule and design.
  - **Product Risk Example**: Failure of a purchased component. Affects software performance.
  - **Business Risk Example**: Competitive product launch. Impacts sales and market assumptions.

Risk Management

Risk	Affects	Description
Staff turnover	Project	Experienced staff will leave the project before it is finished.
Management change	Project	There will be a change of company management with different priorities.
Hardware unavailability	Project	Hardware that is essential for the project will not be delivered on schedule.
Requirements change	Project and product	There will be a larger number of changes to the requirements than anticipated.
Specification delays	Project and product	Specifications of essential interfaces are not available on schedule.
Size underestimate	Project and product	The size of the system has been underestimated.
Software tool underperformance	Product	Software tools that support the project do not perform as anticipated.
Technology change	Business	The underlying technology on which the system is built is superseded by new technology.
Product competition	Business	A competitive product is marketed before the system is completed.

Risk Management - Process



Risk Management – Risk Identification

- Identifying potential threats to the software engineering process, the software product, and the development organization.
- **Methods:**
  - **Team Brainstorming**: Collaborative effort to identify potential risks.
  - **Experience-Based Identification:** Leveraging past project experiences to anticipate potential issues.
  - **Risk Checklists:** Using pre-defined categories (e.g., estimation, organizational, people, requirements, technology, tools) to guide the identification process.

Risk Management – Risk Identification

- **Pruning the List**: Necessary to focus on the most critical risks and avoid overwhelming the risk management process.

Risk Management – Risk Analysis

- **Qualitative Assessment**: Probability and impact of risks are typically assessed qualitatively (e.g., low, medium, high) rather than quantitatively.
- **Impact Categories:**
  - **Catastrophic:** Threatens project survival.
  - **Serious:** Causes major delays.
  - **Tolerable:** Delays within acceptable limits.
  - **Insignificant:** Minimal impact.

Risk Management – Risk Analysis

- **Probability Categories**: Insignificant, Low, Moderate, High, Very High.
- **Risk Matrix**: A table used to visually represent the probability and impact of identified risks.
- **Prioritization**: Focusing on high-impact, high-probability risks (e.g., "top 10" risks as suggested by Boehm).
- **Dynamic Assessment**: Continuous reassessment of risk probability and impact throughout the project lifecycle.

Risk Management – Risk Planning

- **Proactive Risk Mitigation**: Developing strategies to minimize the impact of identified risks.
- **"What-if" Analysis**: Exploring potential scenarios and their consequences to identify potential risks and develop appropriate responses.
- **Risk Management Strategies**:
  - **Avoidance**: Reducing the likelihood of the risk occurring.
  - **Minimization**: Reducing the impact of the risk if it occurs.
  - **Contingency Planning**: Developing backup plans to address the risk if it materializes.

Risk Management – Risk Monitoring

- **Continuous Assessment**: Risk monitoring is an ongoing process throughout the project lifecycle.
- **Focus:** Tracking changes in risk probability and impact.
- **Data Sources**: Utilizing various factors (e.g., number of requirements changes, team performance, external market trends) to assess risk.
- **Regular Reviews**: Incorporating risk monitoring into regular project management reviews.
- **Dynamic Risk Assessment**: Adjusting risk assessments and mitigation strategies based on new information and changing circumstances.

Risk Management – RMMM Plan

- t's a structured way to deal with potential problems that could happen during a project.
- Analogy - Imagine you're building a house. RMMM is like thinking about what could go wrong (like bad weather, material shortages, or unexpected costs) and making a plan to deal with those problems if they happen.

Risk Management – RMMM Plan

- **Risk Identification**: Spotting potential problems.
- **Risk Analysis**: Figuring out how likely and serious each problem is.
- **Risk Mitigation**: Coming up with ways to prevent or lessen the impact of the problems.
- **Risk Monitoring**: Regularly checking if the problems are still a threat and adjusting your plans as needed.

Risk Management – RMMM Plan

- An RMMM Plan is a ***document*** that outlines **how a project team will identify, analyze, mitigate, monitor, and manage potential risks** that could impact the project's success.

Risk Management – RMMM Plan

- **Risk Identification:**
  - Brainstorming: Team members brainstorm potential risks (e.g., delays, budget overruns, technical issues, resource constraints).
  - Risk Checklists: Using pre-defined categories (e.g., technical, schedule, financial, personnel) to guide the identification process.
  - SWOT Analysis: Identifying Strengths, Weaknesses, Opportunities, and Threats.

- **Risk Analysis:**
  - Prioritization: Assessing the likelihood and potential impact of each identified risk.
  - Qualitative or Quantitative Analysis: Using methods like risk scoring matrices or probability/impact assessments.
  - Prioritizing High-Impact Risks: Focusing on the most critical risks that could significantly impact the project's success.

Risk Management – RMMM Plan

- **Risk Mitigation:**
- Developing Strategies: Creating proactive plans to reduce the likelihood or impact of identified risks.
  - Mitigation Techniques:
    - Risk Avoidance: Avoiding the risk altogether (e.g., changing the project scope).
    - Risk Mitigation: Reducing the likelihood or impact of the risk (e.g., improving communication, adding buffer time).
    - Risk Transfer: Shifting the risk to another party (e.g., insurance, outsourcing).
    - Risk Acceptance: Accepting the risk and its potential consequences (e.g., for low-impact risks).

Risk Management – RMMM Plan

- **Risk Monitoring and Control:**
  - **Regular Reviews**: Continuously monitoring identified risks and tracking their status.
  - **Identifying New Risks**: Proactively looking for emerging risks throughout the project lifecycle.
  - **Adjusting Mitigation Plans**: Modifying risk mitigation strategies as needed based on changing circumstances.
  - **Documenting Risk Responses**: Recording all risk-related decisions and actions taken.

Risk Management – RMMM Plan - RIS

Risk information sheet			
Risk ID: P02-4-32	Date: 5/9/02	Prob: 80%	Impact: high

Description:
Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Refinement/context:
Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards.
Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components.
Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.

Mitigation/monitoring:
1. Contact third party to determine conformance with design standards.
2. Press for interface standards completion; consider component structure when deciding on interface protocol.
3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.

Management/contingency plan/trigger:
RE computed to be $20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly.
Trigger: Mitigation steps unproductive as of 7/1/02

Current status:
5/12/02: Mitigation steps initiated.

Originator: D. Gagne	Assigned: B. Laster

Cost and Estimates

- The process of predicting the cost of developing, maintaining, and delivering a software system
- It involves estimating the *resources, time, and effort* required to complete a software project.
- The goal of cost estimation is to provide a *realistic and accurate estimate* of the costs involved in software development so that project managers and stakeholders can make *informed decisions* about project planning, budgeting, and resource allocation.

Cost and Estimates - Aspects

Resource Estimation	• Estimating the number of people, equipment, and materials required to complete the project.
Time Estimation	• Estimating the duration of the project, including the time required for development, testing, and deployment.
Effort Estimation	• Estimating the amount of work required to complete the project, usually measured in person-hours or person-months.
Cost Estimation	• Estimating the total cost of the project, including labor, equipment, materials, and overheads.

Cost and Estimates - Experience-based techniques

- Rely on the manager's experience of past projects and the application domain
- Informed judgment is used to estimate effort requirements
- Typically involves:
  - Identifying project deliverables and software components
  - Estimating effort for each component individually
  - Computing the total effort required
  - Involving a group of people to iterate towards an agreed estimate

Cost and Estimates - Algorithmic cost modeling

- Uses a formulaic approach to compute project effort
- Based on estimates of product attributes, such as:
  - Size
  - Process characteristics
  - Experience of staff involved

Cost and Estimates – Estimation of Lines of Code (LOC)

- To estimate the number of lines of code (LOC) in a project -
  - Break down the project: Divide the project into smaller pieces, called piece_i.
  - Estimate size of each piece: For each piece_i, estimate:
    - *max_i*: the maximum possible size
    - *min_i*: the minimum possible size
    - *best_i*: the "best guess" size
  - Calculate the estimated size: Use the following formula to estimate the size of the whole project:

$$\boldsymbol{Estimated\ Size} = \frac{1}{6} * ((\sum \mathbf{max_}\boldsymbol{i} + \sum \mathbf{min_}\boldsymbol{i} + 4) * \sum \boldsymbol{best_i})$$

where Σ denotes the sum of the values for all piece_i.

Cost and Estimates – Estimation of Lines of Code (LOC)

- A way to estimate the uncertainty of the LOC estimate. The below equations measures how much the actual LOC may deviate from the estimated LOC.

$$\boldsymbol{Standard\ Deviation = \sqrt{sd_1^2 + sd_2^2 + \cdots + sd_n^2}}$$

- The standard deviation of each piece (sd_i) is calculated as

$$\boldsymbol{Standard\ Deviation\ of\ sd_i = \frac{\max_i - \min_i}{6}}$$

Cost and Estimates – LOC based Cost Estimation

- To estimate the cost of a software project based on its size, measured in thousands of lines of code (KLOC).

$$\textbf{Cost} = \alpha * \left(KLOC^{\beta}\right) + \gamma$$

Where:

α (alpha): marginal cost per KLOC, representing the added cost for an additional thousand lines of code

β (beta): exponent reflecting the nonlinearity of the relationship between cost and size

- β > 1: diseconomy of scale (cost per KLOC increases with project size)
- β < 1: economy of scale (cost per KLOC decreases with project size)

γ (gamma): fixed cost of doing any project, representing the initial cost that is independent of project size

Cost and Estimates – Constructive Cost Model (COCOMO)

- It categorizes software projects into three types: ***organic, semidetached, and embedded***, based on the characteristics of the software, development team, and environment.
- **Organic:**
  - Well-understood application
  - Small development team
  - Team members have prior experience with similar projects
  - Examples: application programs, data processing programs

Cost and Estimates – Constructive Cost Model (COCOMO)

- **Semidetached:**
  - Mixed team with experienced and inexperienced staff
  - Team members have some experience with the type of system
  - Examples: utility programs, such as compilers and linkers
- **Embedded:**
  - Software strongly related to machine hardware
  - Large development team
  - Examples: operating systems, real-time system programs

Cost and Estimates – Constructive Cost Model (COCOMO)

- Estimation Stages
  - **Basic COCOMO**:
    - **Estimates project effort and cost based on the project's size and type.**
  - **Intermediate COCOMO**:
    - **Refines the estimate by considering additional factors, such as the development team's experience and the project's complexity.**
  - **Complete COCOMO**:
    - **Provides a detailed estimate by considering all relevant factors, including the project's requirements, design, and testing.**

Cost and Estimates – Constructive Cost Model (COCOMO)

- **Stage 1 – Basic COCOMO Model**

Effort: Effort = a * $(KDSI)^b$ Person-Months (PM)

Development Time: T_{dev} = 2.5 * $(Effort)^c$ Months

Parameters:

- **KDSI**: Estimated size of the software in Kilo Delivered Source Instructions
- **a, b, c**: Constants determined by the category of software project
- **Effort**: Total effort required for software development in Person-Months (PM)
- **T_{dev}**: Estimated time required to develop the software in months

Cost and Estimates – Constructive Cost Model (COCOMO)

- Constants for Different Project Categories

Software Project	**a**	**b**	**c**
Organic	2.4	1.05	0.38
Semi-detached	3.0	1.12	0.35
Embedded	3.6	1.20	0.32

Cost and Estimates – Constructive Cost Model (COCOMO)

- Stage 2 - Intermediate COCOMO Model
- **Cost Drivers (Multipliers):**
  - The Intermediate COCOMO model **defines 15 cost drivers**, which are attributes that affect the development process.
  - Each attribute can be **rated on a six-point scale**, ranging from "very low" to "extra high", to determine its relative importance.
- **Effort Adjustment Factor (EAF):**
  - The product of the effort multipliers of all 15 attributes gives the Effort Adjustment Factor (EAF).
  - The EAF is used to refine the estimates obtained from the Basic COCOMO model.

Cost and Estimates – Constructive Cost Model (COCOMO)

- Stage 2 - Intermediate COCOMO Model – Refining Estimates

Corrected Effort: $\text{Effort}|_{\text{corrected}} = \text{Effort} * \text{EAF}$

Corrected Development Time: $\text{Tdev}|_{\text{corrected}} = 2.5 * (\text{Effort}|_{\text{corrected}})^{c}$

Cost and Estimates – Constructive Cost Model (COCOMO)

- Stage 2 - Intermediate COCOMO Model – 15 Driver Attributes

Cost Drivers	Ratings					
	Very Low	Low	Nominal	High	Very High	Extra High
Product attributes						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.94	1.00	1.08	1.16	
Complexity of the product	0.70	0.85	1.00	1.15	1.30	1.65
Hardware attributes						
Run-time performance constraints			1.00	1.11	1.30	1.66
Memory constraints			1.00	1.06	1.21	1.56
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30	
Required turnabout time		0.87	1.00	1.07	1.15	
Personnel attributes						
Analyst capability	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Software engineer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
Project attributes						
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82	
Use of software tools	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

Cost and Estimates – Constructive Cost Model (COCOMO)

- **Stage 3 – Complete COCOMO**
- The Complete COCOMO model takes into account the following factors to provide a more accurate estimate of project metrics:
  - **Sub-systems**: The model recognizes that a software system is composed of multiple sub-systems, each with its own size, complexity, and cost drivers.
  - **Heterogeneity**: The model acknowledges that different sub-systems may have different characteristics, such as different programming languages, databases, or hardware platforms.
  - **Reusability**: The model considers the possibility of reusing existing components or services, rather than developing them from scratch.

Cost and Estimates – Halstead's Complexity Metrics

- Halstead's complexity metrics provide a way to ***measure the complexity of a computer program*** based on its operators and operands. The metrics are calculated using four parameters:
  - **n1**: Number of unique operators used in the program
  - **n2**: Number of unique operands used in the program
  - **N1**: Total number of operators used in the program
  - **N2**: Total number of operands used in the program

Cost and Estimates – Halstead's Complexity Metrics

Metrics Calculation

- Program Length: $N = N1 + N2$
- Program Vocabulary: $n = n1 + n2$
- Volume: $V = N * \lg n$
- Difficulty: $D = (n1 * N2) / (2 * n2)$
- Effort: $E = D * V$
- Time to Implement: $T = E / 18$ (in seconds)

Cost and Estimates – Halstead's Complexity Metrics

- ***Program Volume (V)***: Represents the minimum number of bits needed to encode the program, taking into account the programming language.
- ***Difficulty (D)***: Indicates how difficult a program is to write or understand.
- ***Effort (E)***: Denotes the "mental effort" required to develop the software or recreate it in another programming language.
- ***Time to Implement (T)***: Estimates the time required to implement the program in seconds.

Cost and Estimates – Function Point Analysis

- **Function Points**
  - Function points are a measure of the functionality required for a project, focusing on the ***external behavior that requires processing***. The classic items to count when calculating function points are:
    - **Inputs**: Items of application data supplied to the program, considered as a single logical input.
    - **Outputs**: Displays of application data, such as reports, screen displays, or error messages, counted as a single output.
    - **Inquiries**: Request-response pairs that do not change internal data, counted as a single inquiry.
    - Internal Files: Logical files maintained by the system, counted as separate files if they contain different types of data.
    - External Interfaces: Data shared with other programs, counted as an interface in each system

Cost and Estimates – Function Point Analysis

- **Key Processing Factors:**
  - Data Communications: Does the system involve data communication with other systems?
  - Distributed Processing: Are the system's functions distributed across multiple hardware locations?
  - Performance Objectives: Are there specific performance objectives that must be met?
  - Heavily Used Configuration: Does the system require a heavily used configuration, impacting design considerations?
  - Transaction Rate: Is the system expected to handle a high volume of transactions?
  - On-Line Data Entry: Is data entered directly into the system by end-users?
  - End-User Efficiency: Are there specific requirements for end-user efficiency?

Cost and Estimates – Function Point Analysis

- **Key Processing Factors:**
  - On-Line Updates: Are there on-line updates to master files?
  - Complex Processing Logic: Does the system involve complex processing logic?
  - Reusability: Is there an intent to reuse parts of the system in other applications?
  - Installation Ease: Are there specific requirements for ease of installation?
  - Operational Ease: Are there specific requirements for ease of operation?
  - Multiple Sites: Will the system be used at multiple locations?
  - Facilitate Change: Is the system designed to accommodate future changes easily?

Cost and Estimates – Function Point Analysis

- **Counting Rules:**
  - Individual fields are not usually counted separately for inputs, outputs, and inquiries.
  - Multiple lines in a report or screen display are counted as a single output.
  - Summary lines may be counted as separate outputs, depending on the authority.
  - Internal files are counted as separate files if they contain different types of data.
  - External interfaces are counted in each system that shares the data.

Cost and Estimates – Function Point Analysis

- **Example:**
  - Input of personal data for an employee: 1 input
  - Report displaying employee data with multiple lines: 1 output
  - Request for an employee's address: 1 inquiry
  - Personnel file with 1000 entries: 1 internal file
  - Personnel file shared with human resources and payroll: 2 external interfaces

Cost and Estimates – Function Point Analysis

- Step 1 (Table to be referred)

Parameter	Count	Weight Factor Low/Medium/high	Total Low/Medium/high
External Inputs (EI)	x	3/4/6	3x/4x/6x
External Outputs (EO)	y	4/5/7	4y/5y/7y
External Inquiries (EQ)	z	3/4/6	3z/4z/6z
Internal Logic Files (ILF)	a	7/10/15	7a/10a/15a
External Logic Files (ELF)	b	5/7/10	5b/7b/10b

TC/UFP = (3x/4x/6x) + (4y/5y/7y) + (3z/4z/6z) + (7a/10a/15a) + (5b/7b/10b)

Cost and Estimates – Function Point Analysis

- **Step 2: Calculate Function Point (FP)**
  - Calculate the sum of all processing factors (Xi).
  - Use the formula: **CAF = 0.65 + 0.01 * ∑ (Xi)**
  - **FP = UFP/TC * CAF**

- **Step 3**: Calculate Productivity, Documentation, and Cost per Function
  - Use the calculated Function Point (FP) value to determine productivity, documentation, and cost per function.

Cost and Estimates – Function Point Analysis

- Scenario Example

Calculate the function point, productivity, documentation, and cost per function for software application with multiple Processing Factors 5, 1, 0, 4, 3, 5, 4, 3, 4, 5, 2, 3, 4, 2.

- The number of EI(Avg): 22,
- The number of EO(Low): 45,
- The number of EI(High): 06,
- The number of ILF(Avg): 05,
- The number of ELF(Low): 02,
- Effort:37 MM,
- Software technical documents: 250 pages,
- User related documents: 120 pages and
- Budgeting/Cost: $7520 per month.

Cost and Estimates – Function Point Analysis

- Scenario Example Solution

Step 1 - Calculate the Total Count of technical complexity(TC):

External Inputs (EI): 22 x 4 (Avg) = 88

External Outputs (EO): 45 x 4 (Low) = 180

External Inquiries (EQ): 6 x 6 (High) = 36

Internal Logic Files (ILF): 5 x 10 (Avg) = 50

External Logic Files (ELF): 2 x 5 (Low) = 10

TC = 88 + 180 + 36 + 50 + 10 = 364

Cost and Estimates – Function Point Analysis

- Scenario Example Solution

Step 2 - Calculate the sum of Processing Factors (Xi):

$$Xi = 5 + 1 + 0 + 4 + 3 + 5 + 4 + 3 + 4 + 5 + 2 + 3 + 4 + 2 = 45$$

Step 3 - Calculate the Function Point (FP):

$$FP = TC \times [0.65 + 0.01 \times Xi]$$

$$FP = 364 \times [0.65 + 0.01 \times 45]$$

$$FP = 364 \times [0.65 + 0.45]$$

$$FP = 364 \times 1.10$$

$$FP = 400.4$$

Cost and Estimates – Function Point Analysis

- Scenario Example Solution

Step 4 - Productivity Calculation

Productivity: Effort (in person-months) / Function Point

Productivity = 37 MM / 400.4 FP

Productivity ≈ 0.0925 person-months/FP

Cost and Estimates – Function Point Analysis

- Scenario Example Solution

Step 5 - Documentation Calculation

Total Documentation: Software technical documents + User related documents

Total Documentation = 250 pages + 120 pages

Total Documentation = 370 pages

Documentation per Function Point: Total Documentation / Function Point

Documentation per Function Point = 370 pages / 400.4 FP

Documentation per Function Point ≈ 0.924 pages/FP

Cost and Estimates – Function Point Analysis

- Scenario Example Solution

Step 5 - Cost per Function Calculation

Cost per Function Point: Budgeting/Cost (in $/month) / Function Point

Cost per Function Point = $7520/month / 400.4 FP

Cost per Function Point ≈ $18.76/FP

Function Point Analysis - Formulas

- **Unadjusted Function Points (UFP)**

- UFP = ∑(Weight of each function type * Count of that function type)

- Function Types:
  - External Inputs (EI)
  - External Outputs (EO)
  - External Inquiries (EQ)
  - Internal Logical Files (ILF)
  - External Interface Files (EIF)

Function Point Analysis - Formulas

- **Value Adjustment Factor (VAF)**

- VAF = 0.65 + (0.01 * ∑(Degree of Influence of each General System Characteristic))

- General System Characteristics (GSCs): 14 factors that influence the system's complexity. Each GSC is rated on a scale of 0 to 5.

Function Point Analysis - Formulas

- **Adjusted Function Points (AFP)**

- AFP = UFP * VAF

Managing People – Dynamics and Communication

- People are the most valuable assets in a software organization.
- Recruiting and retaining good engineers is expensive.
- Productivity is maximized when employees are respected and given appropriate responsibilities.
- Software engineers often excel technically but may lack management skills.
- Project managers must be aware of the challenges of people management.
- Consistency in treatment is crucial for all team members.
- Managers should respect individual skills and differences within the team.
- Early judgments about team members should be avoided.

Managing People – Dynamics and Communication

- **Inclusion:**
  - Creating an environment where everyone's views are valued, regardless of experience, is crucial for effective team contribution.
  - Active listening and consideration of all proposals are key to fostering inclusion.
- **Honesty:**
  - Open and honest communication about both successes and failures within the team builds trust.
  - Acknowledging and respecting the technical expertise of team members, even when it exceeds your own, is essential.
  - Attempting to conceal ignorance or problems will ultimately erode trust and respect within the team.

Managing People – Dynamics and Communication

- Effective project management relies heavily on motivating team members to deliver their best work.
- **Social Needs:**
  - Foster team interaction through physical co-location when possible.
  - Utilize social networking tools and teleconferencing for remote teams (but acknowledge their limitations for building rapport).
  - Organize in-person meetings early in projects to establish connections.

Managing People – Dynamics and Communication

- **Esteem Needs:**
  - Publicly acknowledge team accomplishments.
  - Ensure compensation reflects skills and experience.

- **Self-Realization Needs:**
  - Delegate ownership and responsibility for tasks.
  - Assign challenging but achievable goals.
  - Provide opportunities for training and skill development.

Managing People – Dynamics and Communication

- **Task-Oriented People:**
- **Driven by the work itself**: These individuals find intrinsic motivation in the intellectual challenges and problem-solving aspects of software development.
- **Focus on the technical aspects**: They enjoy the process of designing, coding, and debugging, finding satisfaction in creating elegant and effective solutions.
- Example: A software engineer who thrives on the challenge of optimizing algorithms or developing innovative new features.

Managing People – Dynamics and Communication

- **Self-Oriented People:**
  - **Motivated by personal success and recognition**: Their primary goal is often personal advancement and achieving their own aspirations.
  - **View software development as a means to an end**: They are driven by factors like career progression, financial rewards, and personal prestige.
  - Example: An engineer who is highly motivated to advance their career by taking on challenging projects and demonstrating their skills.

Managing People – Dynamics and Communication

- **Interaction-Oriented People:**
  - Motivated by social interaction and collaboration: They thrive in team environments and find satisfaction in working with others.
  - Value teamwork and communication: They enjoy the social aspects of the workplace and find fulfillment in collaborating with colleagues to achieve shared goals.
  - Increasingly important in modern software development: With the rise of Agile methodologies and user-centric design, collaboration and communication are highly valued.
  - Example: A software engineer who enjoys pair programming, actively participates in team discussions, and values feedback from colleagues.

Managing People – Dynamics and Communication

- **Group Organization**
  - **Technical leadership**: Should the project manager also be the technical leader, or should a separate role (e.g., system architect) be appointed?
  - **Decision-making**: Who will be involved in critical technical decisions (project manager, system architect, or wider team)? How will decisions be made (consensus, individual, etc.)?
  - **External interaction**: Who will handle interactions with external stakeholders and senior management (project manager, system architect, or dedicated liaison role)?
  - **Remote teams**: How can groups integrate people who are not co-located (working from home, different organizations)?
  - **Knowledge sharing**: How can knowledge be shared effectively across the group, avoiding information overload?

Managing People – Dynamics and Communication

- **Types of group organization:**
- **Informal groups:**
  - Common in small programming groups and Agile development teams.
  - Decisions made by consensus, good for experienced and competent teams.
  - Can be problematic for inexperienced or incompetent teams due to lack of direction.
- **Hierarchical groups:**
  - Clear hierarchy with a leader at the top who has formal authority.
  - Decisions made at the top and implemented by lower levels.
  - Communication primarily flows top-down.
  - Well-suited for well-defined problems that can be broken down into components.
  - Not ideal for complex software development projects due to limitations in communication and responsiveness to change.

Managing People – Dynamics and Communication

- Group Communication
  - **Communication is Crucial for Project Success:**
  - **Information Exchange:** Seamless flow of information regarding work progress, design decisions, and changes is paramount.
  - **Problem Resolution:** Effective communication facilitates the resolution of issues that arise within the team and with external stakeholders.
  - **Stakeholder Management:** Keeping stakeholders informed about project progress, changes, and potential roadblocks is crucial for maintaining trust and support.
  - **Team Cohesion:** Open and honest communication fosters a sense of camaraderie and trust within the team, leading to improved collaboration and motivation.

Managing People – Dynamics and Communication

- **Factors Influencing Communication Effectiveness:**
- **Group Size:** As team size increases, the number of communication channels grows exponentially, potentially leading to information silos and miscommunication.
- **Group Structure:** Hierarchical structures can hinder communication by creating barriers between levels and limiting the flow of information. Informal structures generally promote better communication.
- **Group Composition:** Personality clashes, gender imbalances, and varying communication styles can impact the effectiveness of information exchange.
- **Physical Work Environment:** The physical workspace significantly influences communication. Open-plan offices can facilitate interaction, while private spaces are essential for focused work.
- **Communication Channels:** The choice of communication channels (meetings, email, instant messaging, wikis, etc.) impacts the speed, efficiency, and effectiveness of information exchange.

Managing People – Dynamics and Communication

- **Strategies for Effective Communication:**
- **Two-Way Communication**: Encourage open dialogue, active listening, and feedback mechanisms to ensure that information is not only transmitted but also received and understood.
- **Informal Interactions**: Foster informal interactions, such as coffee breaks and team lunches, to encourage casual conversations and build relationships.
- **Embrace Modern Tools**: Utilize tools like wikis, blogs, instant messaging, and video conferencing to facilitate communication within distributed teams.
- **Minimize Barriers**: Reduce hierarchical barriers and encourage open communication across all levels of the organization.
- **Address Communication Challenges**: Proactively identify and address communication issues within the team, such as personality clashes or information overload.

Managing People – Dynamics and Communication

Metrics and Measurements

- Metrics are **specific, quantifiable measures** used to **assess various aspects** of a software project.
- They can be used to evaluate the effectiveness of the development process, the quality of the software product, and the performance of the project team

Metrics and Measurements

- Types of Metrics
  - Project
  - Product
  - Process

Metrics and Measurements

- **Product Metrics**
  - These metrics **focus** on the **characteristics of the software product itself**.
  - Product metrics measure the **characteristics, quality, and performance of a software product or system**. These metrics focus on the **product's features, functionality, and user experience**.
  - Examples include:
    - **Lines of Code (LOC):** A measure of the size of the software codebase.
    - **Function Points:** A measure of the functionality of the software.
    - **Defect Density:** The number of defects per unit of code or functionality.
    - **Code Coverage:** The percentage of code executed by test cases.

Metrics and Measurements

- **Process Metrics**:
  - Process metrics measure the **effectiveness and efficiency of the software development process**. These metrics focus on the **activities, workflows, and practices** used to develop and maintain the software..
  - Examples include:
    - **Cycle Time**: The time taken to complete a specific task or phase of the project.
    - **Defect Removal Efficiency**: The percentage of defects found and fixed during development.
    - **Cost of Quality:** The cost associated with preventing, detecting, and correcting defects.
    - **Team Velocity**: The amount of work a development team can complete in a given time.

Metrics and Measurements

- **Project Metrics**
  - Project metrics measure the **progress, performance, and success of a software development project**. These metrics focus on the project's **scope, schedule, budget, and resources**.

  - Examples include
    - **Schedule Variance**: The difference between the planned and actual project schedule.
    - **Cost Variance:** The difference between the planned and actual project budget.
    - **Risk Exposure**: The potential impact of identified risks on the project.
    - **Customer Satisfaction:** The level of satisfaction of the project's stakeholders.

Metrics and Measurements - Examples

Product Metrics

- Lines of Code (LOC)
- Cyclomatic Complexity
- Defect Density
- Test Coverage
- User Satisfaction

Process Metrics

- Cycle Time
- Lead Time
- Deployment Frequency
- Mean Time to Recovery (MTTR)
- Code Review Coverage

Project Metrics

- Project Schedule Performance Index (SPI)
- Project Cost Performance Index (CPI)
- Return on Investment (ROI)
- Earned Value Management (EVM)
- Project Completion Rate